משימה 6 (בונוס) — הבייט האחרון

נושאים: מערכות ספירה, ייצוג מספרים שליליים, `signed` ו־`unsigned`, אופרטורים ביטיים, מסכות והזזות, ASCII ו־XOR, טיפוסים ברוחב קבוע, קבצים בינאריים, Endianness, ושילוב רכיבי תוכנה ב־C.

לפני שמתחילים: זו משימת העשרה ותרגול עצמאית, שאינה חלק מחמש משימות החובה. חלק מהנושאים אינם נלמדים לעומק בקורס. הכנו עבורכם את מדריך הלימוד העצמי: הוא מספק בסיס, דוגמאות ומפת נושאים להשלמה, אך אינו מחליף לימוד ותרגול עצמאיים. מומלץ לעיין בפרק המתאים לפני כל שלב.

תדריך מבצעי

הבארשבעינייטור הושמד, ובאר שבע ניצלה. אלא שהחקירה חשפה שהמתקפה שימשה גם הסחת דעת לניסיון חדירה לפרויקט מסווג המפותח באוניברסיטת בן־גוריון. עדיין לא ברור אם נגנב מידע, מי מסתתר מאחורי הכינוי **סוכן 67**, או היכן נמצא **מורטי הרשע**.

מאז כישלון המבצע מורטי אינו מוכן עוד לסמוך על מערכות התקשורת של פלנקטון. הוא הכריז על פרוטוקול חדש, "מאובטח במיוחד", המבוסס על מספרים בינאריים, קודים הקסדצימליים ומסרים שעברו פעולות ביטים. צוות המודיעין הצליח ליירט את הנתונים, אבל אינו מצליח לקרוא אותם.

> **מורטי הרשע:** הפעם אף אחד לא יפענח את ההודעות שלי.
>
> **סוכן 67:** החלפת את ההצפנה?
>
> **מורטי הרשע:** הרבה יותר טוב. כתבתי את הכול בהקסדצימלי.
>
> **סוכן 67:** זה לא בדיוק—
>
> **מורטי הרשע:** תסמוך עליי.

המשימה שלכם: לבנות כלי ב־C שיפענח את התקשורת, ישחזר רישומי מפגשים ויעביר לכוחות הביטחון מידע אמין על מיקומם של מורטי וסוכן 67. כל שלב ילמד אתכם דרך אחרת לייצג או לעבד מידע במחשב. רק הרצה מלאה ומוצלחת תחשוף את תוכן הפגישה ואת תוצאת המבצע.

קבצים וסביבת עבודה

תיק הראיות האישי למשימה נוצר באמצעות `operation_b7/.` מהתיקייה הראשית, עם אותם פרטי צוות שבהם השתמשתם במשימות הקודמות. אין לערוך את הקבצים שנוצרו:

- `mission6_Bonus/data/mission6_packets.txt` — מנות תקשורת מקודדות ומפתח פענוח.
- `mission6_Bonus/data/mission6_meetings.bin` — ארכיון מפגשים בפורמט בינארי.

קבצים לעריכה: `mission6_Bonus/bitwise.c` והחלקים המסומנים ב־`mission6_Bonus/main.c`. חתימות הפונקציות והמבנים מוגדרים ב־`mission6_Bonus/bitwise.h`; אין לשנות אותם. אין לשנות את קובצי הראיות, הבדיקות, קובצי `Makefile` או את התשתיות שבתיקיית `./support_`.

המשימה מיועדת ל־Linux x86-64 או ל־WSL2 עם GCC ו־GNU Make, כמו משימות החובה. תקן השפה הוא **C11**.

הרצה ובדיקות

מתוך התיקייה `mission6_Bonus/`:

```
make
make test-a
make test-b
make test-c
make test-d
make test-e
make test-f
```

כל בדיקת שלב בוחנת את המימוש הטכני של אותו שלב בנפרד. אין צורך להשלים את כל המשימה כדי להפיק תועלת מהתרגול. לאחר השלמת כל השלבים:

```
make run
```

חשוב: בדיקות השלבים אינן חושפות את מקום המפגש. הממצאים המצטברים, סיום העלילה וקוד ההשלמה `B7-M6-XXXXXXXX` יוצגו רק בהרצה המלאה. הדוגמאות בהמשך נועדו להסביר את פורמט הנתונים ואינן בהכרח הערכים שיופיעו בתיק האישי שלכם.

שלב A — המספרים המסתוריים

חומר רקע: פרקים 1–2 במדריך.

חלק מהראיות מופיעות כמחרוזות של ספרות בבסיסים שונים. קודם עליכם להמיר אותן לערכים מספריים, בלי לבלבל בין המספר עצמו לבין דרך הכתיבה שלו.

ממשו ב־`bitwise.c`:

```
int parse_base_number(const char *text, unsigned base, uint32_t *out);
int format_binary(uint32_t value, unsigned width, char *out, size_t capacity);
```

`parse_base_number` — חוזה הפונקציה:

- `base` חייב להיות 2, 10 או 16. המחרוזת חייבת להכיל לפחות ספרה אחת, וניתן להשתמש באותיות `A`–`F` או `a`–`f` בבסיס 16.
- בבסיס 2 אפשר להוסיף את הקידומת `0b` או `0B`; בבסיס 16 אפשר להוסיף `0x` או `0X`. הקידומת אינה מחליפה ספרות. בבסיס 10 אין קידומת.
- אפסים מובילים מותרים. סימן `+` או `-`, רווחים ותווים שאינם חוקיים בבסיס הנתון אינם מותרים.
- בהצלחה כתבו את הערך ל־`out` והחזירו `1`; אחרת החזירו `0`. דחו מספר שגדול מ־`UINT32_MAX` בלי לבצע גלישה. אין לקרוא מצביעים לא תקינים.
- אין להשתמש בפונקציות המרה מוכנות כגון `strtol`, `strtoul` או `sscanf` לצורך הפענוח.

`format_binary` — חוזה הפונקציה:

- צרו מחרוזת המכילה בדיוק `width` תווים של `0` או `1`, ולאחריהם `\0`.

- `width` חייב להיות בטווח 1–32; כל הביטים הדלוקים של `value` חייבים להיכנס ברוחב הנתון. אין לקצץ ביטים משמעותיים.
- ודאו שהמערך יכול להכיל `width + 1` תווים. החזירו `1` בהצלחה ו־`0` בכישלון. בכישלון, אם `out` ו־`capacity` תקינים, השאירו מחרוזת ריקה.

דוגמה לימודית: `0xF1`, `241` ו־`11110001` הם שלושה ייצוגים של אותו מספר חיובי.

ממצא בהרצה: המערכת מזהה שהערכים שיורטו מכילים מידע מובנה, ולא מספרים אקראיים.

הריצו `make test-a`.

שלב B — המספרים השליליים

חומר רקע: פרקים 3–4 במדריך.

חלק מהנתונים הם ערכים *חתומים*. אותם ביטים עשויים לתאר מספר חיובי גדול או מספר שלילי קטן, בהתאם לאופן הפענוח. כדי לשחזר זמנים מתוך הארכיון נצטרך לפענח גם הפרשי דקות שליליים.

ממשו:

```
int decode_twos_complement(uint32_t raw, unsigned bits, int32_t *out);
int encode_twos_complement(int32_t value, unsigned bits, uint32_t *out);
```

חוזה הפונקציות:

- `bits` חייב להיות בטווח 2–16. השתמשו בייצוג **משלים ל־2 (Two's Complement)** ברוחב המבוקש.
- `decode_twos_complement` מקבלת ערך גולמי שאינו מכיל ביטים מעבר לרוחב `bits`, ומחזירה את הערך החתום המתאים.
- `encode_twos_complement` מקבלת מספר חתום, מוודאת שהוא נכנס בטווח של `bits` ביטים ומחזירה את דפוס הביטים המתאים כמספר לא חתום.
- בהצלחה החזירו `1`, ובקלט לא תקין או במצביע `NULL` החזירו `0`. אל תסתמכו על הזזת מספרים שליליים או על המרה לא מוגדרת/תלוית מימוש של ערך מחוץ לטווח לטיפוס חתום.

דוגמה לימודית: דפוס הביטים `11110001` מייצג `241` כ־8 ביטים לא חתומים, אך `-15` כ־8 ביטים במשלים ל־2.

ממצא בהרצה: משוחזר תיקון זמן בעל סימן, שמסביר מדוע השעה המקורית ביומן עשויה להיות שגויה.

הריצו `make test-b`.

שלב C — הפרוטוקול של מורטי

חומר רקע: פרקים 5–6 במדריך.

מורטי דחס שלושה שדות לתוך מספר בן 16 ביטים. כל מנת תקשורת מכילה סוג הודעה, מיקום של תו בהודעה ובית של נתונים. אין להשתמש בחיתוך מחרוזות כדי לחלץ את השדות: המידע כבר שמור כמספר.

מבנה המנה המפוענחת מוגדר ב־`bitwise.h`:

```
typedef struct {
    uint8_t type;
```

```
    uint8_t index;
    uint8_t payload;
} PacketFields;

int unpack_packet(uint16_t word, PacketFields *out);
int pack_packet(const PacketFields *fields, uint16_t *out);
```

פורמט המנה, מהביט המשמעותי ביותר לביט הפחות משמעותי:

ביטים	משמעות	רוחב
12–15	`type` — ערוץ/סוג הודעה	4 ביטים
8–11	`index` — מיקום התו	4 ביטים
0–7	`payload` — בית נתונים	8 ביטים

חוזה הפונקציות:

- `unpack_packet` מחלצת את שלושת השדות ממספר בן 16 ביטים. כל תבנית של 16 ביטים חוקית; רק `out == NULL` גורם לכישלון.
- `pack_packet` מרכיבה מספר מהשדות. `type` ו־`index` חייבים להיות בטווח 0–15; `payload` הוא בית, בטווח 0–255 מתוקף הטיפוס. מצביעים לא תקינים גורמים לכישלון.
- החזירו `1` בהצלחה ו־`0` בכישלון. אין לשנות את מבנה הקלט ב־`pack_packet`.
- השתמשו בהזזות ובמסכות ביטים. אין להשתמש ב־bit-fields של `struct`, מכיוון שסדרם בזיכרון תלוי מימוש.

דוגמה לימודית: `0x104B` מתפרק ל־`type = 1`, `index = 0`, `payload = 0x4B`.

ממצא בהרצה: זוהה ערוץ תקשורת בין מורטי הרשע לסוכן 67, וסדר המנות שוחזר. תוכן ההודעות עדיין מוצפן.

הריצו `make test-c`.

שלב D — ההודעות המוצפנות

חומר רקע: פרקים 5 ו־7 במדריך.

בית הנתונים בכל מנה עבר פעולת `XOR` עם מפתח בן בית אחד. עליכם לקבץ מנות לפי `type`, לסדר אותן על פי `index`, לפענח את הבתים וליצור מחרוזת תקינה. אין להניח שהמנות מופיעות בסדר הנכון.

ממשו:

```
int decode_channel(const PacketFields *packets, size_t count,
                   uint8_t type, uint8_t key,
                   char *out, size_t capacity);
```

חוזה הפונקציה:

- התייחסו רק למנות ששדה `type` שלהן שווה לערך המבוקש. מנות מערוצים אחרים אינן משנות את ההודעה.
- פענחו כל `payload` באמצעות `payload ^ key`. התוצאה היא תו ASCII, או בית שערכו `0` המציין סוף הודעה.

- דרשו רצף אינדקסים מלא המתחיל ב־0, בלי כפילויות ובלי חורים, ובו תו סיום `\0` יחיד באינדקס האחרון. אין תווים נוספים אחריו. מספר האינדקסים המרבי הוא 16 (0–15).
- כל התווים שלפני הסיום חייבים להיות בתחום ASCII הניתן להדפסה (`0x20`–`0x7E`). ההודעה אינה יכולה להיות ריקה.
- כתבו הודעה מלאה המסתיימת ב־`\0`, רק אם יש מספיק מקום ב־`out`. הצלחה מחזירה `1`; כישלון מחזיר `0` ומשאיר `out` ריק, כאשר אפשר לעשות זאת בבטחה.
- אין להסתמך על סדר המנות במערך ואין לשמור במחרוזת מצביעים למאגר המקורי.

פורמט קובץ המנות: השורה הראשונה היא `KEY=HH`, כאשר `HH` הוא בית הקסדצימלי. כל שורה נוספת מכילה מנת 16 ביטים בארבע ספרות הקסדצימליות. סוגי הודעות שונים מכילים רמזים שונים; אחד מהם כולל מידע הדרוש לשלב הסופי.

ממצא בהרצה: משוחזרים רמזים לכך שהשניים מתכננים מפגש בקמפוס. פרטי המיקום והשעה המדויקים עדיין אינם ידועים.

הריצו `make test-d`.

שלב E — שחזור יומן המפגשים

חומר רקע: פרקים 3, 4 ו־8 במדריך.

בארכיון נוסף נמצאו רשומות בינאריות של פגישות. חלקן בוטלו, חלקן פגומות ואחרות אינן רלוונטיות. הקובץ אינו מכיל טקסט: עליכם לקרוא בתים עם `fread`, לפרש ערכים מרובי־בתים בסדר **Little-Endian**, לזהות מספרים חתומים ולוודא את תקינות הרשומות.

פורמט רשומה: כל רשומה תופסת **10 בתים בדיוק**, ללא רווחים וללא padding:

היסט בבתים	שדה	פירוש
1–0	`meeting_id`	מזהה פגישה, `uint16_t`, Little-Endian
3–2	`base_minute`	מספר דקות מאז חצות, `uint16_t`, Little-Endian
5–4	`location_code`	קוד מיקום, `uint16_t`, Little-Endian
6	`flags`	ביט 0: מאושרת; ביט 1: בוטלה; שאר הביטים חייבים להיות אפס
7	`reserved`	חייב להיות אפס
8	`minute_adjust`	הפרש דקות חתום במשלים ל־2 של 8 ביטים
9	`checksum`	XOR של תשעת הבתים הקודמים (0–8)

השעה האמיתית מחושבת באמצעות `base_minute + minute_adjust`. התוצאה חייבת להיות בטווח 0–1439. פגישה פעילה היא פגישה שבה ביט האישור דלוק וביט הביטול כבוי.

מבנה הנתונים והפונקציות:

```
typedef struct {
    uint16_t meeting_id;
    uint16_t base_minute;
    uint16_t location_code;
    uint8_t flags;
```

```
    int16_t minute_adjust;
    uint16_t final_minute;
} MeetingRecord;

int parse_meeting_record(const uint8_t raw[10], MeetingRecord *out);
int find_confirmed_meeting(FILE *input, MeetingRecord *out);
```

`parse_meeting_record` — חוזה הפונקציה:

- פענחו את הבתים לפי הטבלה, בלי להניח דבר על סדר הבתים של המחשב שעליו רצה התוכנית.
- דרשו `meeting_id > 0`, `location_code > 0`, `base_minute <= 1439`, דגלים תקינים, בית שמור אפס, checksum נכון ושעה סופית תקינה. צירוף של "מאושרת" ו"בוטלה" יחד אינו חוקי.
- החזירו `1` עבור רשומה תקינה ו־`0` אחרת. רשומה תקינה יכולה להיות גם לא מאושרת או מבוטלת — ההחלטה אם היא פעילה שייכת לפונקציה הבאה.

`find_confirmed_meeting` — חוזה הפונקציה:

- קראו את הקובץ ברשומות של עשרה בתים, באמצעות `fread`. התעלמו מרשומות שלמות שפענוחן נכשל, מרשומות שלא אושרו ומרשומות שבוטלו.
- החזירו `1` רק אם נמצאה **בדיוק פגישה פעילה ותקינה אחת**; כתבו אותה ל־`out`. אם אין התאמה, יש יותר מהתאמה אחת, הקובץ נקטע באמצע רשומה או אירעה שגיאת קריאה — החזירו `0`.
- בכישלון, אם `out` תקין, אפסו אותו. דחו מצביעי `NULL`. אין לשנות את קובץ הראיות.
- **אסור** לקרוא ישירות למבנה C באמצעות `fread(&record, sizeof(record), 1, input)`: גודל המבנה, ריפוד השדות וסדר הבתים שלו אינם פורמט הקובץ. קראו ל־`uint8_t raw[10]` ובנו מתוכו את הערכים.

דוגמה לימודית: רשומה שבה זמן הבסיס הוא 09:20 וההפרש הוא `-10` דקות מייצגת זמן פגישה של 09:10. זו דוגמה להבנת החישוב, לא רמז לפתרון האישי.

הממצא שיתקבל בהרצה: מספר הפגישה, **השעה המדויקת**, קוד מיקום שעדיין אינו מפוענח ומצב הפגישה. דוגמת מבנה הפלט:

```
[BINARY ARCHIVE RECOVERED]
[BYTE ORDER: LITTLE-ENDIAN]
[MEETING RECORDS DECODED]

MEETING ID: <decoded id>
TIME: <decoded HH:MM>
LOCATION CODE: <decoded hex code>
STATUS: CONFIRMED

[INTELLIGENCE UPDATE]
The meeting time has been recovered.
The location code remains unresolved.
[LOCATION IDENTIFICATION: PENDING]
```

הריצו `make test-e`.

שלב F — המרדף האחרון

חומר רקע: שילוב כל הפרקים במדריך.

בשלב האחרון אין פורמט מספרי חדש. עליכם לחבר את רכיבי הפענוח שכבר כתבתם: לקרוא את המנות, לשחזר את ערוץ המידע המתאים, לקרוא את יומן המפגשים ולהצליב את **קוד המיקום** עם **שם המיקום** שהוסתר בתקשורת.

ערוץ מידע מיוחד, שמספרו `14` (או `0xE`), מכיל מחרוזת בפורמט:

```
LOC:HHHH=PLACE
```

`HHHH` הן בדיוק ארבע ספרות הקסדצימליות של קוד המיקום, ו־`PLACE` הוא מזהה מיקום המורכב מאותיות לטיניות גדולות. המחרוזת כולה, כולל תו הסיום, חייבת להתאים למגבלת 16 הבתים של ערוץ תקשורת יחיד. אין להניח מראש מהו הקוד או מהו שם המקום בתיק האישי.

ממשו:

```c
int resolve_location(const PacketFields *packets, size_t count,
                     uint8_t key, uint16_t location_code,
                     char *out, size_t capacity);
```

חוזה הפונקציה:

- פענחו את ערוץ `0xE` באמצעות `decode_channel` והמבנה שהוגדר לעיל.
- השוו את קוד המיקום מתוך ההודעה לקוד שהתקבל מארכיון המפגשים. דחו התאמה חסרה, מחרוזת פגומה או שם מקום ריק.
- בהצלחה העתיקו ל־`out` את שם המקום בלבד כמחרוזת C תקינה והחזירו `1`. בכישלון החזירו `0` והשאירו מחרוזת ריקה אם אפשר.
- אין לקבע בקוד את קוד המיקום, את שם המקום, את שעת המפגש או את מפתח הפענוח.

שילוב ב־`main.c`: השלימו את החלקים המסומנים כך שהתוכנית תקרא את שני קובצי הראיות, תשתמש בפונקציות של שלבים A–F, תדחה תיק חקירה שלא אומת במלואו, ותעביר את הממצא הסופי למנגנון הדיווח שסופק. אין להדפיס הודעת הצלחה או קוד השלמה במקום להפעיל את המנגנון עצמו. שחררו משאבים וסגרו קבצים גם במסלולי כישלון.

הריצו `make test-f`, ולאחר מכן:

```
make run
```

רק בהרצה מלאה מוצלחת יוצגו שם מקום המפגש, השעה, התפתחות המבצע ותוצאת המרדף. קוד ההשלמה יוצג בפורמט `B7-M6-XXXXXXXX`.

דוח והגשה

למשימת הבונוס **אין דוח חקירה נוסף**. מטרתה היא תרגול ולמידה של C וייצוג נתונים, ולא כתיבת מסמך. הגישו את קובצי המקור שערכתם: `mission6_Bonus/bitwise.c` ו־`mission6_Bonus/main.c`, בהתאם להנחיות ההגשה של הקורס.

אין להגיש את קובצי הראיות, את `BITWISE_GUIDE.md`, את `bitwise.h`, קובצי הרצה או את `support_/`.